TECHNICAL UNIVERSITY OF MOLDOVA
Faculty of Computers, Informatics, and Microelectronics
Department of Software Engineering
Software Engineering Study Programme

LABORATORY REPORT

Lab 8: The Same Calls, Inside Your Own OS

Operating Systems

Student: Gafenco Victor

Group: FAF-241

Instructor: Assistant lecturer, Patricia Reițman

Chișinău - 2026

CONTENTS

Abstract **3**

Introduction **4**

1 Process creation inside xv6 . . . . . **5**

1.1 Running echo 5

1.2 Replacing echo with ls 5

1.3 Failed execution 6

1.4 Comparison with Linux: observations 6

2 Bidirectional communication with pipes **7**

2.1 Protocol and descriptor ownership 7

2.2 Observed exchanges 7

2.3 Pipe observations 8

Conclusions **9**

ABSTRACT

Two xv6 user programs demonstrate the process lifecycle and communication between a parent and child. Execution experiments compare successful program replacement with a missing executable. A one-byte exchange over two pipes demonstrates blocking reads, descriptor ownership and deterministic message order.

Keywords: xv6, fork, exec, wait, pipe, interprocess communication

INTRODUCTION

Two xv6 user programs exercise process creation and pipe communication. `run` starts a child that becomes `echo`; additional runs replace it with `ls` and attempt to execute a nonexistent program. `pingpong` exchanges one byte between a parent and child using two pipes.

The programs were built against xv6 revision `06aad25c735fd3159bdfae5680be4eab7b1668b2` using the RISC-V GCC toolchain and executed in QEMU with one virtual CPU. Both the kernel and filesystem image built successfully. Complete build and execution output is retained in `evidence/`.

The submitted `Makefile` adds the following tab-indented entries to `UPROGS`:

```
$U/_run\
$U/_pingpong\
```

The corresponding sources are `user/run.c` and `user/pingpong.c`. The existing `args` and `sleep` entries are retained, with their sources included so that the submitted Makefile has all its additional user programs. Copy the Makefile and submitted user sources into the matching xv6 tree, then build and boot with `make qemu`. Exit QEMU with Ctrl-A followed by X before rebuilding a changed program.

1 PROCESS CREATION INSIDE XV6

1.1 Running echo

`run` prints its PID and calls `fork()`. The child prints its own PID and constructs a null-terminated argument vector:

```
char *args[] = { "echo", "hello", "from", "the", "child", 0 };
exec("echo", args);
```

The parent calls `wait(&status)` and prints the collected exit status. The observed output was:

```
$ run
parent: my pid is 3
child: my pid is 4
hello from the child
parent: child 4 ended with status 0
```

The parent and child have different PIDs. After successful `exec`, the child remains process 4 but executes `echo` rather than the instructions of `run`. The final parent message appears after `echo` terminates.

1.2 Replacing echo with ls

The child argument vector and execution call were changed to:

```
char *args[] = { "ls", 0 };
exec("ls", args);
```

After rebuilding and rebooting, selected output was:

```
$ run
parent: my pid is 3
child: my pid is 4
.              1 1 1024
..             1 1 1024
README         2 2 2441
run            2 3 34584
pingpong       2 4 35720
[remaining directory entries omitted]
parent: child 4 ended with status 0
```

The complete listing is in `evidence/ls.txt`. The directory listing confirms that the child executed `ls`, and its normal termination produced status 0. PIDs repeat because these variants were tested in separate boots.

1.3 Failed execution

Changing the execution call to `exec("nosuchprog", args);`, rebuilding and running again produced:

```
$ run
parent: my pid is 3
child: my pid is 4
exec failed
parent: child 4 ended with status 1
```

The error message comes from the code immediately after `exec`. That code is reachable only when execution fails. The child then calls `exit(1)`. The submitted source retains the successful `echo` version.

1.4 Comparison with Linux: observations

1. xv6 uses `fork()`, `exec("echo", args)` and `wait(&status)`. The Linux program uses `fork()`, `execlp("ls", "ls", "-l", (char *)NULL)` and `waitpid(pid, &status, 0)`. Both follow the same sequence: create a child, replace its program, and collect its termination status. xv6's `exec` takes an argument array and does not perform the Linux `execlp` search through `PATH`; its `wait` reaps an available child rather than selecting a PID. There is only one child here, so that distinction does not affect the experiment.
2. The code of `echo` prints "hello from the child". `run` supplies those words as arguments; after successful `exec`, its child-side code has been replaced.
3. The failed-exec status is 1 in xv6 and 127 in the Linux experiment because the programs explicitly call `exit(1)` and `exit(127)`, respectively. These are program choices, not different automatic translations by the kernels. xv6 returns the integer passed to `exit` through `wait`; Linux uses an encoded wait status from which the program extracts `WEXITSTATUS` for a normal exit.

2 BIDIRECTIONAL COMMUNICATION WITH PIPES

2.1 Protocol and descriptor ownership

`pingpong` creates `p2c`, for parent-to-child traffic, and `c2p`, for child-to-parent traffic, before calling `fork`. The parent initializes the byte to `x`. The child receives it, prints its PID and "received ping", sends the same byte back, closes its pipe ends and exits. The parent receives and checks the returned byte, prints its PID and "received pong", and waits for the child.

Process	Reads from	Writes to	Unused ends closed immediately
Parent	`c2p[0]`	`p2c[1]`	`p2c[0]`, `c2p[1]`
Child	`p2c[0]`	`c2p[1]`	`p2c[1]`, `c2p[0]`

The essential operations are `read(fd, &b, 1)` and `write(fd, &b, 1)`. Their return values must equal one for a successful one-byte exchange. The implementation checks pipe creation, `fork`, reads, writes and the child's exit status. It closes each used end after its final operation. On an error exit, xv6 also closes the process's remaining descriptors.

The ordering is enforced by data dependencies. The child prints before sending the reply, and the parent prints only after reading that reply. No sleep is required.

2.2 Observed exchanges

Three consecutive executions in the same boot produced:

```
$ pingpong
6: received ping
5: received pong
$ pingpong
8: received ping
7: received pong
$ pingpong
10: received ping
9: received pong
```

Every execution printed exactly one ping followed by one pong, then returned to the shell. The ping PID belongs to the child; the pong PID belongs to its parent. The increasing values reflect newly created processes, not a fixed requirement of the protocol.

2.3 Pipe observations

1. Two pipes provide two separate one-way channels. A pipe is a shared byte stream, not an addressed message connection. If both processes read and write the same pipe without an additional synchronization protocol, the parent can consume the byte it just sent instead of a reply. Separate pipes make the sender and receiver unambiguous in each direction.
2. `read(c2p[0], &b, 1)` makes the parent wait for the reply when the return pipe is empty and a writer remains open. The later `wait(&status)` reaps the child; it is not the operation that waits for the reply byte. If the child has already written the byte before the parent calls `read`, that read can complete immediately.

CONCLUSIONS

The xv6 experiments reproduce the same process lifecycle used on Linux with a smaller user API. Successful `exec` preserves process identity while changing the executing program, and failed execution returns to code that chooses an exit status. The ping-pong program demonstrates that blocking reads synchronize processes through data availability. Separate pipes, deliberate descriptor ownership and a final wait provide an ordered exchange and proper child cleanup.